NPU 芯片设计调研报告

基本原理、技术演进与架构取舍
从模型执行到系统吞吐

技术路线发展 • 25 篇代表论文 • 设计取舍与边界

2014—2026 年代表性进展

调研日期：2026年10月19日

先理解 NPU 的任务与执行过程，再分析架构取舍。
包含背景引入、章节关系与代表论文。

阅读导航

阅读背景：NPU 的任务、结构与设计目标 3

阅读地图：从模型执行过程理解各章关系 5

全景判断：峰值算力不等于有效吞吐 6

1. 架构全景：从单一乘加阵列到全栈系统瓶颈 7

2. 技术演进史：底层架构的“瓶颈迁移”路径 8

3. 数据流与阵列：数据复用与计算利用率 9

4. 量化与稀疏的代价：低比特不等于绝对加速 10

5. 大模型推理：Prefill与Decode的硬件博弈 11

6. 近存与存内计算：距离缩短了，但外围开销仍在 12

7. 多芯封装与互连：同步与通信的木桶效应 13

8. 全栈设计与评估：先搜映射，再做RTL 14

9. 路线选型：按工作负载的等待类型匹配架构 15

10. 核心论文阅读地图：按问题分类的25篇文献 16

资料范围与证据说明 19

文内青色文字可点击打开原始来源。文末按问题列出论文，并区分首次预印本年份与正式发表年份。

阅读背景：NPU 的任务、结构与设计目标

NPU 是什么，为什么需要它

NPU 是 Neural Processing Unit 的缩写，通常译为神经网络处理器。它是一类面向神经网络计算的专用加速器。它可以作为手机等系统芯片中的一个模块，也可以采用独立加速芯片的形式。本报告采用较宽的研究口径，将 Google TPU 这类名称不同、同样面向神经网络的专用处理器纳入讨论；各产品并不因此具有相同结构。Arm，Edge AI 技术说明；Sze 等，2017

以图像分类为例：应用接收一张照片，模型将像素转换为多层数值计算，最后输出各类别的分数。模型中的参数称为权重；每层计算产生、继续传给后续层的数据称为激活。训练根据数据调整权重，推理使用训练得到的权重处理新输入。本文主要围绕推理解释设计取舍，也讨论涉及训练的系统架构；两类任务的硬件需求需要分别判断。Sze 等，2017，§II–III

神经网络中的许多运算反复执行“输入乘权重，再累加”。例如，输入为 2、3、1，权重为 0.4、0.1、0.7，一次加权求和就是 $2\times0.4+3\times0.1+1\times0.7=1.8$。这是解释乘加的算术示例，不是完整模型：真实网络还包括非线性、归一化等处理。大量输入和输出组合起来，就形成后文反复出现的矩阵与张量计算。本报告算例；运算结构参见 Sze 等，2017，§II–III

中央处理器（CPU）能执行这些计算，图形处理器（GPU）也能并行处理。NPU 的设计机会来自重复性：针对常用运算安排专门的计算单元和数据路径，争取用更少时间或能量完成任务。这类似于批量处理表格时，提前固定计算公式与数据布局；准备工作可以复用，但表格结构改变后，原有安排未必合适。专用程度提高，也会收窄高效执行的任务范围。Sze 等，2017，§V

CPU、GPU、NPU 不是按先进程度排列的三级产品。 CPU 通常负责系统控制与通用任务，GPU 提供并行计算能力，NPU 针对神经网络运算进行专门设计；实际系统可以协作使用这些资源。某个模型是否适合 NPU，要检查算子支持、形状、精度和软件实现，不能仅凭设备名称判断。Arm，异构 AI 技术说明；Google，TPU 使用说明

模型怎样在 NPU 上执行

将训练好的模型交给 NPU，仍需要一套执行过程。以下以数字推理加速器 NVDLA 为例；不同 NPU 的模块和接口会变化，但这个例子足以说明计算、存储与软件为何必须一起讨论。NVIDIA, NVDLA Primer

- **安排计算。** 编译器把模型转换为硬件能执行的操作，决定数据布局，以及过大的计算如何拆成小块；运行时与驱动提交任务，处理操作之间的依赖。
- **读取数据。** 输入和权重从系统内存进入芯片缓冲区。缓冲区容量有限，计算需要的数据可能分多次进入。
- **执行运算。** 乘加阵列将许多乘法与累加单元组织在一起，并行处理规则计算；其他单元完成激活、池化或数据变换。具体分工取决于架构。
- **传递结果。** 中间结果交给下一操作，或写回内存。若后续计算再次需要已经移出的数据，就会增加一次读取。

这里的片上 SRAM 是容量较小的本地存储；DRAM 是常见系统内存，HBM 是高带宽内存的一种实现。它们影响数据能放多少、多久能够到达计算单元。**计算单元、存储和软件共同决定一次模型执行的速度。** NVDLA 的设计说明因此同时讨论乘加单元数量、缓冲容量与内存带宽，而非只讨论计算阵列。NVIDIA, NVDLA 系统集成与软件设计

NPU 芯片设计需要确定什么

设计者需要先确定目标任务：运行哪些模型，采用什么数值精度，允许多长响应时间，功耗与面积各有多少预算；再选择计算单元、存储、数据路径、互连和软件接口。TOPS 表示每秒万亿次运算，是描述计算能力的一类指标；它还需要说明位宽、运算计数方式及是否采用稀疏口径。它不能独立描述完整模型的执行时间。Sze 等，2017，§VIII；Jouppi 等，2017

本报告把 NPU 设计理解为：在质量、时间、能耗和硬件成本约束下，组织模型的计算与数据。 后文主要分析架构和系统层面的选择；晶体管电路、工艺制造和完整后端实现不在本文展开。读者不需要先掌握 RTL，但需要从这里开始同时关注计算发生在哪里、数据如何到达，以及各环节怎样衔接。

阅读地图：从模型执行过程理解各章关系

前面的执行过程给出了全文的共同对象：一个模型从输入到输出，要经过计算安排、数据读取、单元执行和结果传递。以下各章分别检查其中的不同问题，并不属于同一层级的替代路线。例如，量化可以与阵列设计结合，编译优化也可以用于采用 PIM 的系统。

内容位置	它接着回答什么问题	与整体执行过程的关系
全景判断、第 1 章	为什么计算单元很快，完整模型仍可能很慢	建立计算、存储、软件共同决定性能的系统视角
第 2 章：技术演进	研究重点为何从专用计算扩展到存储与系统	按瓶颈变化串联历史，帮助理解论文的设计背景
第 3 章：数据流与阵列	计算怎样分配，数据在哪里重复使用	改变计算单元的占用，以及数据访问次数
第 4 章：量化与稀疏	每个数要保留多少位，哪些运算可以省略	改变存储量与执行工作，同时引入格式处理和误差控制
第 5 章：大模型推理	输入处理、逐步生成与动态路由有何不同	检查任务变化怎样移动计算、访存与调度瓶颈
第 6、7 章：PIM 与多芯	计算和存储怎样靠近，系统怎样扩展	改变物理放置与连接方式，也改变转换、通信和同步成本
第 8、9 章：评估与选型	怎样比较上述方案，并决定资源投入方向	将工作负载、执行映射、硬件与证据放在一起判断
第 10 章：论文地图	需要深入某个问题时应读哪些工作	提供对应的机制、实验与资料入口

因此，全文的阅读顺序是：**先理解 NPU 在执行什么，再看执行为何受限，随后比较架构改动，最后检查改动是否带来系统收益。** 第一次接触 NPU，可先读背景、全景判断与第 1 章；已有架构基础，则可按问题直接进入后续章节。这是本报告的组织与阅读建议。

全景判断：峰值算力不等于有效吞吐

前文解释了模型如何在 NPU 上执行。接下来需要检查：计算阵列的能力能有多少转化为完整任务的速度。

增加计算资源，并不能保证提升任务的整体速度。 Google 第一代 TPU 论文给出一个具体例子：同一块标称 92 TOPS 的芯片，运行 CNN0 时达到 86 TOPS，运行 CNN1 时为 14.1 TOPS。硬件没有变化，工作负载变化后，实际吞吐出现了明显差异。Jouppi 等，2017，表 3

论文的计数器分析将原因进一步拆开：CNN1 执行矩阵运算的周期不到一半；部分层未能充分占用乘加单元；同时存在等待权重的时间。这些损失涉及不同环节，不能统一归结为“带宽不足”。Jouppi 等，2017，§4

后文持续检查三个问题：计算单元是否有足够的合适任务，数据能否及时到达，以及局部优化增加了哪些系统开销。第 3 章展开利用率与数据流，第 4 至 7 章检查不同优化及其代价，第 8、9 章讨论如何验证和选择。

架构决策首先需要解释现有性能损失。 目标若是规则的大矩阵、数据供给充足，增加计算资源可能有效；如果主要时间消耗在访存、调度或通信，则需要改变对应环节。TPU 的同芯片测量说明，这一判断必须针对具体工作负载。Jouppi 等，2017

1. 架构全景：从单一乘加阵列到全栈系统瓶颈

本章沿着背景中介绍的执行过程，逐项检查计算、存储和软件的分工。先理解系统分工，后面的技术路线才有明确位置。

矩阵乘法很快，完整模型还是慢。这种情况并不矛盾：模型还要做归一化、激活、重排和控制；加速器还要取数据、启动任务、和主处理器交接。Gemmini 研究的起点就是：把加速器单独拎出来测，会漏掉 SoC 争用、操作系统和软件栈的开销。Genc 等，2021

先看清一颗 NPU 的分工。矩阵单元集中做规则的乘加；向量单元处理逐元素运算等工作；标量单元处理地址、分支与调度。昇腾公开文档里的 Cube、Vector、Scalar 就是这种分工。它们的名字不用背，真正要问的是：**离开矩阵单元之后，模型还要在哪里花时间？**华为 CANN 文档，2026

再看数据。片上 SRAM 放得近、容量有限；外部 DRAM/HBM 能放更多东西，数据却要经过接口搬进来。DMA 负责搬运，片上网络负责连接各单元。计算器就在手边，每查一个数字却得离开座位去取一趟资料，算得快也救不了交卷时间。NVDLA 设置专门的卷积缓冲区，就是为了减少反复访问系统内存。NVDLA Primer

要检查的地方	别只问配置，直接问损失	资料入口
矩阵与向量单元	大矩阵很快，小算子和非线性要等多久？	TPU，Jouppi 等，2017；昇腾公开架构
SRAM与外存	哪些数据反复读取？哪一级先堵住？	Eyeriss，Chen 等，2016
数值格式与稀疏	少算的乘法，是否换成了更多解码和查表？	EIE，Han 等，2016；FP8，Micikevicius 等，2022
编译与运行时	切块合不合适？有没有CPU回退与频繁交接？	Gemmini，Genc 等，2021
互连与物理实现	是单元在算，还是整套系统在等数据、等同步？	Simba，Shao 等，2019；TPU v4，Jouppi 等，2023

这些模块不是新名词清单。它们决定了矩阵结果能否顺利进入下一步。NVDLA 甚至为逐点处理、池化等运算设置独立模块，说明“乘加之外怎么办”本来就是芯片设计的一部分。NVDLA 架构文档

本报告把 NPU 用作神经网络专用加速器的统称，以数字架构为主。GPU 的 Tensor Core 用来解释相邻的数值与稀疏机制，PIM 和模拟存内计算单独讨论；它们不能因为都跑神经网络，就被当成同一种硬件。Sze 等，2017；NVIDIA，2020

完整模型的耗时需要覆盖矩阵计算之外的处理、搬运与交接。 各环节可以通过融合或并行减少开销，仍需在系统层面测量结果。 固定任务可以采用更专的电路；模型会持续变化的产品，则需要保留可编程处理能力。只要目标包含这些变化，就不能只拿矩阵峰值证明设计合理。Genc 等，2021；华为 CANN 文档，2026

2. 技术演进史：底层架构的“瓶颈迁移”路径

明确一颗 NPU 包含哪些环节后，再看研究为何先后关注这些环节。本章按瓶颈及其解决方式组织历史；不同路线会交叉发展，不能将时间顺序理解为完全替代。

2014年的 DianNao 给出了一个直接的答案：神经网络里有大量重复的运算，没必要总让通用处理器承担全部执行开销。把常用计算和局部缓冲专门做出来，能在较小面积里提供较高吞吐。这是专用架构的吸引力，也是路线的起点之一。Tianshi Chen 等，2014

接下来的难题更难绕开：乘加做快以后，数字从哪里来？Eyeriss 把重点放到数据复用；TPU 把大矩阵阵列和软件管理存储放在一起。再往后，EIE 和 SCNN 开始直接处理压缩、稀疏的数据。**先减少通用执行开销，再减少搬运，再减少无用计算。** 这条脉络能解释这些论文为什么出现，也比背架构名称有用。Chen 等，2016；Jouppi 等，2017；Han 等，2016；Parashar 等，2017

但省事的新办法，还会带来新麻烦。压缩数据需要解码；小网络未必装得满大阵列；多颗芯片要相互通信。Eyeriss v2 处理模型形状变化，Simba 处理跨 Chiplet 的任务和数据分配，Gemmini 则要求把系统开销一起测进去。研究没有离开“算得快”，只是发现它越来越不能单独成立。Chen 等，2019；Shao 等，2019；Genc 等，2021

时间与代表工作	原来卡在哪里	改了什么	又多了什么代价
2014：DianNao	通用处理器执行神经网络的代价高	专用计算、局部缓冲	模型变大以后，片外数据仍要搬
2016—2017：Eyeriss、TPU	算得动，供数却跟不上	空间阵列、复用、软件管理存储	要让层形状与硬件匹配
2016—2019：EIE、SCNN、Eyeriss v2	零值占资源，紧凑网络装不满阵列	压缩执行、跳零、灵活互连	索引、分配和负载均衡更复杂
2019—2021：Simba、Gemmini	单块计算模块的结果代表不了整机	Chiplet、全栈集成与评估	通信、系统争用都得计时
2022—2024：FP8、FlashAttention、PagedAttention	大模型压住了数值、访存和缓存系统	改格式、改算子、改缓存管理	硬件内核和运行时必须跟上
2024—2026：NeuPIMs、P3-LLM、TPU 8t/8i	不同推理阶段、训练与服务的需求分化	NPU与PIM协作，按任务专化	转换、切换与跨单元协作会吃掉收益

后两行的证据入口：FP8，Micikevicius 等，2022；FlashAttention，Dao 等，2022；PagedAttention，Woosuk Kwon 等，2023；NeuPIMs，Heo 等，2024；P3-LLM，Yuzong Chen 等，2026；TPU 8t/8i，Google，2026。

新路线值得看，是因为它解决了旧路线正在付出的代价，不是因为年份更新。 这些路线仍在并行使用；大矩阵、低批量推理和稀疏网络不会共享一个永远最优的设计。Timeloop 的跨架构研究也明确观察到了这种取舍。Parashar 等，2019

3. 数据流与阵列：数据复用与计算利用率

技术演进中反复出现的第一个架构问题，是计算与数据如何匹配。本章讨论阵列怎样接到任务、数据怎样在不同存储层级复用；这为下一章理解压缩后的执行代价提供基础。

利用率诊断：区分供数不足与工作负载失配

计算单元在等数据，增加带宽可能有用；数据已经到了，矩阵形状却只用得上少量单元，增加带宽就治错病了。TPU 论文对 CNN1 的分析同时查出了供数等待和阵列装不满，说明这两种损失确实可能叠在一起。Jouppi 等，2017，§4

拿一个映射示例算账：256×256的阵列，一次任务只能用16行，又不能合并其他任务，行方向的占用就是16/256＝6.25%。其余93.75%的行没有替这次计算干活。**这时再扩阵列，扩大的是闲置能力。** 这里算的是假设映射的瞬时行占用，不是某款芯片的实测利用率，也不能据此说93.75%的功耗都浪费了。本报告示例；层形状与利用率的机制见 Chen 等，2019

Eyeriss v2 值得接在这里读。紧凑、稀疏的网络虽然运算少了，层与层的形状和复用机会却更不一样。作者为此设计分层网格网络，让数据分配适应不同层。**模型压小以后，原来按大层设计的阵列未必更舒服，反而可能更难用满。**Chen 等，2019

脉动阵列：局部传递与流水执行

脉动阵列把乘加单元按规则连起来，操作数按节拍从相邻单元经过，计算与传递交叠进行。数据到了一个单元就参与乘加，再按设计的数据流继续传递或积累。省掉的是一部分反复远距离取数的代价，换来的是对数据到达顺序、矩阵形状和节拍的要求。TPU v1 是这条路线的代表。Jouppi 等，2017

这像多人接力核账：每个人处理自己负责的一项，再把需要的数字交给邻座。人数多，只有在任务能连续分配时才有用；中途断档，后面的工位照样要等。机制依据：Jouppi 等，2017

Eyeriss：以多级数据复用减少访存

卷积里，同一个权重会乘上多个输入，同一个输入也会参与多次运算。Eyeriss 的行驻留数据流，把权重、输入和部分和的复用一起考虑，尽量让数据在较近的位置多用几次。这不是往芯片上塞一块更大的存储就结束了，还要决定计算分给谁、数据在哪里留多久、部分结果怎么累加。Chen 等，2016

权重驻留就是尽量留下权重，输出驻留就是尽量在本地累加输出；行驻留把卷积行之间的多种复用组织起来。选择哪一种，要看真正减少了哪一级的存取。**驻留的是名字，省下的是访问次数。只写“采用某某数据流”，性能账还没开始算。**Chen 等，2016

先证明现有单元为什么没忙起来，再批准下一批乘法器。 大而规则、数据充足的矩阵适合密集阵列；小矩阵、形状多变或供数不足时，要先处理分区、切块、任务合并和数据复用。若这些问题已经解决，扩阵列才有明确的用处。Jouppi 等，2017；Chen 等，2019

4. 量化与稀疏的代价：低比特不等于绝对加速

上一章在给定数据表示下调整计算与搬运。本章进一步改变数据表示和需要执行的工作：量化减少数值位宽，稀疏利用零值减少部分运算。收益仍要回到实际执行路径检查。

低比特执行：压缩收益与解码开销

“32位改成4位，快8倍。”这个算法只算了位数，没算机器到底执行什么。AWQ 同时做量化方法和 TinyChat 推理实现，原因就在这里：低比特权重需要合适的打包和内核，压缩率自己不会变成加速比。Lin 等，2024

W4A16 里，W4说权重存成4位，A16说激活是16位。它没有承诺乘法器、累加器和所有中间结果都按4位运行。常见执行路径还要从压紧的数据中取出权重，结合缩放因子恢复计算所需的数值，再参与运算；高效内核会把这几步融合，避免额外写回一整份高精度权重。**解码成本可能被藏好，也可能把省下的时间吃回去，要看实现。** Lin 等，2024

压缩文件很小，但不能拿着压缩包直接编辑里面的文档。要么先解压，要么软件边读边解压。量化也有这层区别：存储格式变紧凑了，执行路径还得有人打通。不过量化通常会引入误差，普通无损压缩不会；压缩率和答案质量还得分开检查。Lin 等，2024

量化误差：异常值与敏感通道的处理

把数值范围粗暴切成少量刻度，少数异常大的激活会把刻度拉得很开，其他值就分不清了。SmoothQuant 通过等价变换，把激活里的部分量化困难转移到权重；AWQ 根据激活统计保护敏感权重通道。两篇论文都在处理“哪些误差不能乱省”，不是只换一个数据类型。Xiao 等，2023；Lin 等，2024

FP8 也有取舍。E4M3 给更多尾数位，E5M2 给更多指数位；用同样的8位，分别争取更细刻度或更大范围。微缩放格式让一组数共享尺度，还要携带组尺度和处理转换。设计者省掉了数值本身的位数，也就接下了误差控制和额外元数据的工作。Micikevicius 等，2022；Darvish Rouhani 等，2023

稀疏加速：跳零收益受索引与非矩阵开销约束

零值可以跳过，但跳之前得知道它在哪里，跳之后还得把结果累加到正确位置。EIE 直接处理压缩稀疏网络，SCNN 同时利用权重和激活的稀疏性，都是把这条执行路径专门做出来。SCNN 在其 CNN 架构评估中，相对配置相当的稠密方案报告约2.7倍性能和2.3倍能效；这个结果有网络与硬件条件，不能只拿“稀疏率”推出来。Han 等，2016；Parashar 等，2017

NVIDIA 的2:4结构化稀疏，用“每四个值中两个为零”的规整限制，降低硬件追踪非零位置的难度。符合条件的计算有最高两倍吞吐机会，整模型却还包含其他操作。若矩阵部分原占60%，即便它快两倍，总加速也只有 $1/(0.4+0.6/2)\approx1.43$ 倍。这是明确假设下的推算，不是某张卡的测试成绩。NVIDIA，2021；系统开销背景：Genc 等，2021

量化只交出更小的权重文件，就只证明省了存储；要宣称提速，还得交出目标硬件上的执行结果。 带宽受限时，缩小权重有很大机会；如果转换、索引或其他算子已经主导时间，压缩率再漂亮，也不能替整模型报喜。质量、内核和计时范围必须一起给出。Lin 等，2024；Genc 等，2021

5. 大模型推理：Prefill与Decode的硬件博弈

前两章解释了硬件能够怎样优化，本章转向任务本身。大模型的不同阶段会改变矩阵形状、数据复用和调度需求，因此同一套硬件不一定在所有阶段都高效。

Decode：低批量下的权重带宽约束

Prefill 一次处理提示词中的许多 token，通常更容易形成大矩阵计算。低批量 Decode 每步只为少量新 token 做计算，权重却还要被读取；增加批量能让更多请求共享权重，但也会改变延迟和调度。NeuPIMs 据此区分适合 NPU 的矩阵矩阵运算和适合 PIM 的带宽密集运算。Heo 等，2024

前面70B模型的算例，最刺眼的地方就在这里：FP16权重是140 GB，4位理论上降到35 GB。假设每步都从外存读完、有效带宽1 TB/s、batch=1，只搬权重的时间就分别至少是140毫秒和35毫秒，对应约7.1与28.6 token/s的乐观上限。**当这笔读取躲不掉时，峰值乘加再翻倍，也不会让数据提前到达。** 算例未包含KV、量化元数据、计算或通信；权重常驻片上、高批量复用充分时，要重新算账。本报告推算；机制依据：Heo 等，2024

FlashAttention：通过分块与在线归一化减少HBM访问

标准注意力计算容易产生很大的中间矩阵。FlashAttention 把计算切块，维护必要的归一化统计，减少 HBM 与片上 SRAM 之间的往返。它抓住的是数据流，论文还报告了对应 GPU 实验。Dao 等，2022

做分组统计时，读一批数据就更新累计结果，没必要先把每一项抄成另一整本账，再从头统计。FlashAttention 的难点是：分块以后，仍要正确完成注意力的归一化。它没有省掉稠密注意力所有成对计算，也没有承诺不同浮点执行顺序逐位一致。**少写中间结果，才是这篇论文给芯片设计者的直接提醒。**Dao 等，2022

PagedAttention：以分页管理减少KV缓存浪费

KV 缓存存的是历史 token 的键和值。请求长度会变，缓存也跟着长；如果为每个请求预留大块连续空间，就可能出现“有人占着用不完，别人却申请不到”的浪费。PagedAttention 借用操作系统分页的办法，按块管理KV，支持共享。作者在当时的模型和服务配置下，报告同等延迟下相对 FasterTransformer、Orca 约2—4倍吞吐。Woosuk Kwon 等，2023

活页本按需加页，比给每个人提前留一本厚本子更省空间；但没有省掉已经写下的内容。分页主要处理分配浪费和共享，不会自动让所有KV数值变小。用一组假设算：32层、8个KV头、每头128维、FP16、8,192个token，单请求KV就是2×32×8×128×2×8,192＝1 GiB；32个请求是32 GiB。MLA、滑动窗口或KV量化会改变这笔账，不能照搬此式。本报告推算；分页机制：Kwon 等，2023；MLA实例：DeepSeek-AI，2024/2025

MoE：稀疏激活仍需承担专家存储与路由成本

DeepSeek-V3 有671B总参数，每个token激活37B。这里省的是每个token需要参与的部分计算，并没有把全部专家的存储需求一起删掉。专家分布到多颗芯片之后，路由、数据交换和负载不均仍要处理。DeepSeek-AI，2024/2025

NPUMoE 的 Apple M 系列实验把麻烦写得更具体：专家路由带来动态形状，top-k和scatter/gather不规整，小专家任务太多还要付启动与同步开销。作者把静态、规整计算交给NPU，把动态部分留给CPU/GPU。这里的经验是**先分清哪部分适合NPU，再追求卸载；把整模型硬塞进去，可能只是增加交接。** 论文是2026年预印本，平台结果有明确边界。Benazir 与 Lin，2026

只用一个大矩阵跑分替大模型选芯片，漏掉的恰好可能是业务最慢的部分。 先分别测Prefill、Decode、KV和动态操作，再讨论阵列配置。低批量、高批量、长上下文与MoE的瓶颈会移动，不能把“Decode必然只受带宽限制”写成定律。Heo 等，2024；Benazir 与 Lin，2026

6. 近存与存内计算：距离缩短了，但外围开销仍在

当任务仍主要受数据搬运限制时，另一种选择是改变计算与存储的位置关系。本章讨论近存与存内计算；它们可以与量化、调度优化结合，并非替代前面所有设计。

数字PIM：并行调度与访存争用

PIM 要解决的问题很直接：数据大部分在内存里，反复搬到处理器再算，为什么不把一部分计算放近一点？但它不是一个统一电路。数字PIM把计算单元放在内存内部或附近；模拟存内计算让器件与电路直接承担部分数值运算。二者的接口、精度和成本不同。Heo 等，2024；Le Gallo 等，2023

普通访存和PIM计算可能争用同一套资源，NPU也可能在等PIM结束。NeuPIMs 为此设计双行缓冲和子批量交错，争取让两边并行。它相对NPU-only约2.4倍吞吐的评估结果，来自具体协作设计。**“接上一块PIM”与“让PIM带来净收益”，中间还隔着调度和访存冲突。**Heo 等，2024

P3-LLM：数值格式与PIM计算单元的协同设计

P3-LLM 没有只要求PIM多算一点。作者把低精度格式、PIM计算单元和融合执行一起设计，试图在面积约束下提高吞吐。这条路线抓住了一个容易漏掉的代价：为了少搬数据而塞进内存的计算电路，本身也要占面积、耗功率。Yuzong Chen 等，2026

论文使用RTL综合和周期级系统仿真，硬件速度比较聚焦4K上下文的Decode；相对HBM-PIM、Ecco、Pimba，报告平均约4.9、2.0、3.4倍加速。这里可以讨论架构潜力，不能据此承诺一台真实手机或服务器的端到端速度。**它解决了哪些成本，和这些成本是否已经在实物上验证，是两个问题。**Yuzong Chen 等，2026，§VI-A

模拟存内计算：阵列能效与外围开销

模拟交叉阵列用电导表示权重，以电压和电流完成矩阵向量运算。数字输入要进入这套电路，结果也要读回数字系统；数模转换、模数转换、噪声和器件差异都得处理。Le Gallo 等的64核相变存储混合信号芯片，已经把证据推进到实物层面，阅读时尤其要看外围与数字通信部分。Le Gallo 等，2023

只报阵列内部的能效，像只算机票、不算去机场和转机的时间。航程可以很短，整趟旅程仍然可能很长。模拟阵列的收益也要把转换、校准、数字后处理和传输放回去算；这不否定阵列的价值，只是决定整机到底拿到多少。机制依据：Le Gallo 等，2023

PIM要交付的是少花总时间、少耗总能量，不是少跨一次接口的漂亮示意图。 低复用、带宽受限的工作更有机会；复用提高、计算变重、频繁更新权重或精度要求变严时，收益可能缩小。NPU/PIM分工和外围代价必须按目标任务重新测。Heo 等，2024；Yuzong Chen 等，2026；Le Gallo 等，2023

7. 多芯封装与互连：同步与通信的木桶效应

上一章讨论计算怎样靠近数据。本章讨论单芯片资源不足时，如何将任务扩展到多芯片；跨芯片通信与同步因此进入执行路径。

单颗芯片不够大，就用多颗小芯片拼起来。这条路能扩展计算和存储，但跨芯片的数据不会瞬间到达。Simba的36 Chiplet实物原型报告系统最高128 TOPS、最高6.1 TOPS/W；论文同样花力气解决数据放置、任务切分和尾部延迟。Shao 等，2019

最容易低估的是同步：一层计算要等各部分到齐，某颗Chiplet算得少但通信远，也可能拖住整层。开会时十个人等最后一份材料，前九份提前到达也不能开始讨论。增加计算资源，只有在数据和任务分得合适时才会变成有效速度。机制依据：Shao 等，2019

TPU v4 把光电路交换用于重构系统互连。它改变的是芯片之间怎么连接；神经网络乘加仍由电子计算单元执行。别因为看见一个“光”字，就把光互连和光计算写成同一条技术路线。Jouppi 等，2023

产品路线：HBM容量与互连拓扑的任务专化

Ironwood在2025年公布时，给出每芯片192 GB HBM、约7.37 TB/s带宽。到2026年，Google 公布的TPU 8t与8i在存储和互连上进一步分化：8t为216 GB HBM、128 MB片上SRAM；8i为288 GB HBM、384 MB SRAM。这里沿用官方的GB、MB单位。Google，2025；Google，2026

8t使用3D torus，8i使用Boardfly并加入集合通信加速引擎。与其盯着两款芯片谁的峰值更大，我更看重这个选择：厂商已经把训练与低延迟推理的差异，落实到了SRAM、HBM和通信组织上。这是基于公布架构的判断；实际业务收益仍要看模型、延迟目标和测试范围，公布也不等于全部配置已经普遍可用。Google，2026

多颗芯片的峰值可以相加，业务速度得等通信和同步同意。 Chiplet、板级系统和集群不是同一种距离；封装、链路、任务划分、散热与功耗都要随规模核对。Simba是实物证据，TPU新产品说明是厂商资料，二者都支持研究互连，却不能拿来拼一张统一跑分表。Shao 等，2019；Google，2026

8. 全栈设计与评估：先搜映射，再做RTL

前面各章给出了不同层面的设计选择。现在需要将它们放到同一工作负载下评估：先检查执行映射，再逐步补充系统与物理实现证据。

映射探索：先评估执行方案，再修改硬件

同一硬件、同一层，换种切块和数据复用方式，能耗就可能大幅变化。Timeloop 给过一个足够扎眼的例子：在类似NVDLA的1024-MAC架构上运行VGG的一层，约48万种映射的性能都在峰值5%以内，能效却能相差近19倍。这是论文分析模型中的案例，不是48万次芯片实测。Parashar 等，2019，图1与§II

只盯吞吐，你甚至可能看不出哪种执行方式在多花电。 Timeloop 对架构和映射一起搜索；MAESTRO 从数据复用与资源占用估算代价；Accelergy 把计算、存储和互连的动作转换成能量估计。它们的用途是尽早筛掉不划算的方案，而不是给尚未实现的芯片盖上“已经省电”的章。Parashar 等，2019；Hyoukjun Kwon 等，2019；Wu 等，2019

验证流程：从工作负载到物理实现

下面是本报告的实施建议。它把Gemmini的全栈评估、Timeloop/MAESTRO的映射探索、Accelergy的能量估计和Simba的实现验证串成一条工作路径；不表示这些实验已经在本工作区完成。Genc 等，2021；Shao 等，2019

步骤	要拿出来的东西	这一关要挡住什么返工
1. 定工作负载	模型版本、形状、批量、上下文、质量与延迟目标	设计完才发现业务根本不是那种大矩阵
2. 查等待	算子耗时、访存量、阵列占用、CPU回退和通信	把缺带宽、装不满、启动慢混成一个问题
3. 搜映射	切块、循环顺序、驻留、跨核划分	软件能解决的问题，被误判为要改电路
4. 算完整能量	计算、SRAM、DRAM、互连动作及单位能量	只算乘法器，把搬运和外围漏掉
5. 跑全栈原型	参考实现、周期模型、RTL、驱动与编译路径	模块很快，整模型却卡在交接与回退
6. 过物理实现	面积、时序、布线、功耗、供电与持续负载	逻辑图很漂亮，落到版图却达不到指标

工具与方法入口：Timeloop，2019；MAESTRO，2019；Accelergy，2019；Gemmini，2021；NVDLA硬件文档；Simba，2019。

对EDA尤其要留一层清醒：更灵活的网络要付出选择和连线的代价，更大的存储也要占地方。架构模型里的一次访问，落到物理实现里有距离、延迟和功耗。这里是工程推论；Simba对不均匀通信延迟的处理，提供了可以具体检查的案例。Shao 等，2019

模拟器像设计阶段的计算草稿，能帮你提前发现账算错，却不能替代最终验收。分析模型用于大范围筛选，周期仿真检查时序行为，RTL综合与物理实现收紧成本，实物测量回答真实系统能做到多少。P3-LLM和Simba分别走到了不同证据层级，结论也必须停在各自证据能支撑的位置。Yuzong Chen 等，2026；Shao 等，2019

别用流片或后端返工，替一个本可先查清的映射问题付学费。 早期探索可以大胆，用小模型快速排错；但要宣称频率、能效或可制造性，就得补上对应的实现证据。仿真有仿真的价值，越过它的能力边界才是问题。Parashar 等，2019；Genc 等，2021

9. 路线选型：按工作负载的等待类型匹配架构

本章把前文的机制与评估要求用于路线选择。选型依据是已确认的性能限制，以及质量、延迟和成本约束；表中各方向可以组合使用。

目标业务、批量和质量要求还没定，就别急着争“脉动阵列还是PIM更先进”。Timeloop已经表明，映射和硬件的好坏要跟工作负载一起判断。Parashar 等，2019

确认存在的瓶颈	钱和精力先花在哪里	什么情况下要改主意	依据
稳定CNN的反复访存	复用、SRAM分配、量化与融合	模型形状变化使原映射失效	Eyeriss；Eyeriss v2
规则大矩阵的计算需求	数字阵列、合适的数值格式与供数能力	非矩阵或通信已占主要时间	FP8；TPU v4
低批量LLM的权重读取	压缩、带宽，条件合适再评估PIM	批量增加或转换成本压过节省	AWQ；NeuPIMs；P3-LLM
长上下文的注意力访存与KV空间	注意力内核、KV布局和服务调度	模型已改变缓存形式或缺少高效硬件实现	FlashAttention；PagedAttention
MoE的动态形状与专家不均	专家布局、调度、小任务与异构协作	路由变化使静态估计失效	DeepSeek-V3；NPUMoE
固定模型的低能耗推理	评估模拟存内计算及全部外围	精度、转换、校准或更新代价过高	Le Gallo 等，2023

表中建议来自对应论文的机制分析，完整出处见第10章；没有做统一平台复测。

比较口径：精度、稀疏、负载、延迟、功耗与证据层级

先问精度和质量。 INT4、INT8、FP8不是同一种工作；模型答错了，吞吐再高也没完成原任务。Micikevicius 等，2022

再问密集还是稀疏。 EIE同时报告压缩执行能力与对应的等效稠密吞吐，两种数字回答的问题不同，不能挑大的当真实执行量。Han 等，2016

接着问批量、上下文和延迟目标。 吞吐可以靠攒请求提高，用户却可能等得更久；TPU v1已经把99分位响应时间当作业务约束，PagedAttention也在同等延迟下比较服务吞吐。Jouppi 等，2017；Kwon 等，2023

然后问功耗算到哪里，以及结果从哪里来。 阵列、芯片、板卡和整机是不同范围。P3-LLM用仿真与综合估计硬件结果，Simba测实物原型；图表旁边必须保留这些条件。Yuzong Chen 等，2026；Shao 等，2019

本报告建议：CNN报告质量、batch=1延迟和单次推理能量；LLM报告质量、首token与后续token延迟、吞吐、峰值内存和尾延迟。用完整模型计时，别只挑最漂亮的内核。建议依据：Jouppi 等，2017；Kwon 等，2023；Genc 等，2021

先解释瓶颈、给出完整计时，再谈路线优劣。 单算子结果适合判断局部潜力；只有当质量、工作负载和系统范围都对齐，它才可能支持产品或架构选择。证据还没到那一步，就把结论停在那里。Genc 等，2021

10. 核心论文阅读地图：按问题分类的25篇文献

前文已经建立问题与架构之间的关系。本章据此整理文献入口，便于读者从某个具体问题继续深入；阅读顺序不代表论文或芯片的性能排名。

先读P1、P3、P4，弄清计算为什么会等数据；再读P16、P17、P19，看看映射与系统开销怎样改变结果；接着读P12、P13、P21、P24，跟到注意力、KV和NPU/PIM协作。这是按问题安排的阅读顺序。下面保留完整题名，方便回到原文查条件。

基础与经典数字架构

- P1 | Sze、Chen、Yang、Emer，2017。 Efficient Processing of Deep Neural Networks: A Tutorial and Survey，Proceedings of the IEEE。问题：计算、存储与数据复用分别花什么代价？先用它弄清计算、存储和数据复用；它写于2017年，别用它代替今天的LLM服务研究。原文
- P2 | Tianshi Chen 等，2014。 DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning，ASPLOS。问题：怎样把常见神经网络计算做成小型专用加速器？证据：论文设计与评估；别把历史基准外推到今天。原文
- P3 | Yu-Hsin Chen 等，2016。 Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks，ISCA。问题：如何同时复用权重、输入和部分和？证据：数据流分析与架构评估；重点是 CNN。原文
- P4 | Jouppi 等，2017。 In-Datacenter Performance Analysis of a Tensor Processing Unit，ISCA。问题：真实业务里峰值与延迟约束如何冲突？证据：已部署 TPU 的测量；比较基线属于当时。原文
- P5 | Han 等，2016。 EIE: Efficient Inference Engine on Compressed Deep Neural Network，ISCA。问题：压缩后的稀疏矩阵怎样直接计算？证据：硬件设计与评估；不要把等效稠密吞吐当作执行吞吐。原文
- P6 | Parashar 等，2017。 SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks，ISCA。问题：权重与激活都稀疏时，数据如何送到乘法器？证据：架构评估；索引、累加与负载平衡是阅读重点。作者机构页面
- P7 | Yu-Hsin Chen 等，2019。 Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices，IEEE JETCAS；预印本 2018。问题：网络变小、形状变多之后，旧架构为什么失配？用途：理解灵活性代价。原文

数值格式、注意力与服务

- P8 | Micikevicius 等，2022。 FP8 Formats for Deep Learning，arXiv。问题：8 位浮点如何分配范围和精度？证据：格式定义与模型实验；不代表任意层都可无条件改成 FP8。原文
- P9 | Darvish Rouhani 等，2023。 Microscaling Data Formats for Deep Learning，arXiv。问题：共享尺度能否兼顾低位宽与模型质量？用途：连接数值格式与硬件成本。原文

- P10 | Guangxuan Xiao 等，2023。 SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models，ICML；预印本 2022。问题：如何处理激活异常值，让 W8A8 更可用？证据：量化与实现实验。原文
- P11 | Ji Lin 等，2024。 AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration，MLSys；预印本 2023。问题：怎样保护敏感权重而避免昂贵的混合精度布局？用途：端侧低比特部署。原文
- P12 | Dao 等，2022。 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness，NeurIPS。问题：少写中间结果能否比少算一些更有效？证据：算法分析与 GPU 实验；迁移 NPU 需要重新实现内核。原文
- P13 | Woosuk Kwon 等，2023。 Efficient Memory Management for Large Language Model Serving with PagedAttention，SOSP。问题：KV 碎片与共享如何限制批量？证据：服务系统实验；不是芯片测量论文。原文
- P14 | DeepSeek-AI，2024/2025。 DeepSeek-V3 Technical Report，arXiv，2024 首发、2025 修订。问题：MoE、MLA 与训练系统如何改变硬件需求？用途：真实工作负载背景；不是 NPU 设计论文。原文
- P15 | Benazir 与 Felix Xiaozhu Lin，2026。 Efficient Mixture-of-Experts LLM Inference with Apple Silicon NPUs，arXiv。问题：动态专家路由怎样适配偏静态的 NPU？证据：Apple M 系列实验；平台与预印本状态需保留。原文

设计工具、系统与存内计算

- P16 | Parashar 等，2019。 Timeloop: A Systematic Approach to DNN Accelerator Evaluation，ISPASS。问题：怎样公平比较架构与映射？证据：模型与案例；模型结果不是流片测量。作者机构页面
- P17 | Hyoukjun Kwon 等，2019。 Understanding Reuse, Performance, and Hardware Cost of DNN Dataflows: A Data-Centric Approach，MICRO，工具名 MAESTRO。问题：如何从数据复用估算性能与成本？用途：快速探索，不能代替后端实现。原文
- P18 | Yannan Nellie Wu、Emer、Sze，2019。 Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs，ICCAD。问题：如何把各部件动作转换成能量账？证据：估算方法；依赖组件模型和活动计数。作者机构页面
- P19 | Genc 等，2021。 Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration，DAC；预印本 2019。问题：系统争用和软件开销会怎样改变局部结论？证据：开源生成器、完整系统及芯片实践。原文
- P20 | Yakun Sophia Shao 等，2019。 Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture，MICRO。问题：跨 Chiplet 的延迟不均怎样拖慢整层？证据：36 Chiplet 实物原型；CNN 结果有具体工作负载边界。作者机构页面
- P21 | Heo 等，2024。 NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing，ASPLOS。问题：NPU 和 PIM 如何并行协作？证据：架构评估；重点读子批量调度与访存冲突。原文

- **P22 | Le Gallo 等，2023。** A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference，Nature Electronics。问题：模拟阵列怎样与数字外围构成可用系统？证据：实物芯片；精度、校准和通信都要读。原文
- **P23 | Jouppi 等，2023。** TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings，ISCA。问题：专用计算怎样扩展到系统级互连？证据：已建成系统；光互连不等于光计算。原文
- **P24 | Yuzong Chen 等，2026。** P3-LLM: An Integrated NPU-PIM Accelerator for Edge LLM Inference Using Hybrid Numerical Formats，ISCA 2026；预印本 2025 首发，核对版本为 2026年5月3日 v4。问题：低精度怎样同时改变准确率、PIM 面积与数据流？证据：RTL 综合、周期级系统仿真和模型质量实验。原文与方法
- **P25 | Jonghun Lee 等，2026。** TriGen: NPU Architecture for End-to-End Acceleration of Large Language Models based on SW-HW Co-Design，arXiv。问题：MX 低精度、查表非线性与小片上存储如何联合优化？本报告仅作前沿补充入口，不把摘要中的加速比当作流片事实。原文

工程资料：接口、规格与实现约束

- **D1 | NVIDIA，NVDLA 公开文档。** 架构规范与入门说明：适合看模块、缓冲和编译路径；它的 CNN 算子集不代表现代 LLM NPU 的完整能力。
- **D2 | 华为，CANN 公开文档，页面更新于 2026年9月14日。** AI Core SIMD 编程模型：适合看 Cube、Vector、Scalar、本地存储与不同耦合模式；不要跨型号混用限制。
- **D3 | NVIDIA，2021。** Ampere/TensorRT 稀疏推理说明：用来核对 2:4 的条件和执行路径；GPU 邻近机制，不是 NPU 型号榜单。
- **D4 | Google，2025。** Ironwood 官方发布：用来观察 HBM 容量、带宽与推理取向；厂商数据。
- **D5 | Google，2026年4月22日。** TPU 8t/8i 技术说明：用来观察训练/推理专化、SRAM、HBM 与拓扑选择；公布规格与实际业务收益分别判断。

读到加速比就停，等于只看了结账单上的折扣。还要查原价、条件和漏算了什么。 这25篇论文覆盖不同任务和证据层级，不能横着拼成排名；它们适合用来定位问题、选择验证办法，不代表本报告已经复现了结果。

资料范围与证据说明

本报告沿用2026年10月19日的调研范围。本次修订补充 NPU 定义、基本运算、执行流程、设计目标与章节关系，并核对 Arm 的处理器说明、NVDLA 的执行流程及 Sze 等人的基础综述；后续技术路线继续沿用已有调研。主线是数字NPU的计算、存储、软件和系统设计，另谈数字PIM与模拟存内计算；光计算、脉冲神经网络、器件工艺和制造供应链不在本篇独立展开。

25篇论文与5组工程资料的出处保留。原稿已核对的论文结果继续沿用；这次重写另外核对了TPU原文表3与第4节，以及Timeloop图1与第2节，用来把利用率和映射问题讲具体。P3-LLM已检查全文评估方法；其余资料的阅读深度仍是相关原文、作者机构资料、摘要或官方文档，没有统一做全文复现。

140 GB、6.25%、KV容量与1.43倍等例子都给了假设，是本报告推算。TPU的86与14.1 TOPS是原论文业务代码测量，表3注明不含主机开销；Timeloop的近19倍能效差异是分析模型结果；P3-LLM的硬件速度来自仿真与综合；Google新品规格来自厂商。它们的证据边界不同，正文分别交代。

本文没有重新运行模型、芯片或架构仿真，也没有以写作口吻代替实验。路线选择部分是基于已查资料给出的条件式建议，不能当作已经验证的新方法或本工作区已选定的课题。